Course Review and Career Communication

Three active tickets.
Two assigned tickets.
One report to explain.

Day 1: worked review and one individual GitHub concept guide
Day 2: final demonstrations and interview practice

A system review

One ticket report connects data meaning with operational decisions.

1. **Purpose**
   Which user and workload decision does it support?
2. **Model**
   What does one record mean, and how are relationships represented?
3. **Protect**
   Which bad states and unauthorized actions are refused?
4. **Perform**
   Which query shapes matter, and what measured work supports them?
5. **Recover**
   Which failure is tolerated, and how is restoration verified?
6. **Reproduce**
   Can another person inspect it safely without receiving a secret?

The review notebook

Notebook 09: Four Tickets, Two Different Totals

What runs	What it demonstrates
SQLite inside Python	Real SQL, constraints, rollback, and an isolated restore
Python dictionaries and JSON	The same ticket facts in a document representation
MQL reference on the page	A comparable grouping, not a live Atlas connection

Choose Open in Colab on the weekly page. Run the cells in order.
No packages, database passwords, or cloud database accounts are needed.

The four-ticket example

agent_id	name
201	Priya
202	Noah
203	Elena

One row per staff member.
A staff member can have no tickets.

ticket_id	assignee_id	status
1	201	open
2	201	resolved
3	NULL	new
4	202	open

Active means new, open, or in_progress.
NULL means ticket 3 is unassigned.

Selection and projection in SQL

```
SELECT ticket_id, assignee_id, status
FROM tickets
WHERE status IN ('new', 'open', 'in_progress')
ORDER BY ticket_id;
```

Result IDs: 1, 3, 4. Active-ticket count: 3.

WHERE selects matching rows. SELECT chooses columns.
The result still has one row per ticket.

Rows produced by the left join

Staff	Matched active ticket	Status
Priya	1	open
Noah	4	open
Elena	NULL	NULL

Elena remains because staff is on the left.
Ticket 3 is unassigned, so it does not match a staff member.

A zero-inclusive staff report

```
SELECT s.agent_id, s.name, COUNT(t.ticket_id) AS active_count
FROM staff AS s
LEFT JOIN tickets AS t
    ON t.assignee_id = s.agent_id
   AND t.status IN ('new', 'open', 'in_progress')
GROUP BY s.agent_id, s.name
ORDER BY s.agent_id;
```

The ON condition defines a matching active ticket.
COUNT(t.ticket_id) ignores the NULL placeholder for Elena.

Two totals with different populations

Staff	Active count
Priya	1
Noah	1
Elena	0

Assigned active: 2 **Unassigned active: 1** **All active: 3**

A staff workload report needs a separate unassigned category to describe the full backlog.

Two changes that break the report

Version	Elena	Consequence
COUNT(t.ticket_id), status in ON	0	Includes all staff with their matched-ticket counts
COUNT(*), status in ON	1	Counts Elena's placeholder as a ticket
COUNT(t.ticket_id), status in WHERE	Absent	Filters out Elena's unmatched row

The COUNT(*) version totals 3 by accident.
That does not mean it correctly counts or assigns the active tickets.

A controlled assignment change

```
BEGIN;
UPDATE tickets SET assignee_id = 203 WHERE ticket_id = 3;
-- Run the staff report and active-ticket query again.
ROLLBACK;
```

During the change: Priya 1, Noah 1, Elena 1. All active: 3.
After rollback: Priya 1, Noah 1, Elena 0. All active: 3.

SQL outline. The notebook binds a target ID and guarantees rollback in finally.

A rejected value and a rolled-back assignment

Step in the notebook transaction	What happens
Assign ticket 3 to Elena	The valid assignment succeeds provisionally
Set status to finished	The CHECK constraint rejects the value
ROLLBACK in finally	The earlier assignment is also discarded
Read ticket 3 again	assignee_id is NULL and status is new

The explicit rollback undoes the earlier successful statement.
The failed SQLite statement alone does not establish that result.

The same ticket as JSON

```
{
  "_id": 3,
  "assignee_id": null,
  "status": "new"
}
```

The notebook converts all four tickets to JSON.
Each document still means one ticket. assignee_id remains a reference.
Changing the representation does not assign ticket 3 to a staff member.

Grouping tickets in MongoDB

```
db.tickets.aggregate([
  { $match: { status: { $in: ["new", "open", "in_progress"] } } },
  { $group: { _id: "$assignee_id", active_count: { $sum: 1 } } }
])
```

Groups: 201 has 1, 202 has 1, null has 1.
There is no Elena group. Group order is unspecified.

Reference for a collection containing the four sample documents.
The notebook runs a Python grouping model, not MongoDB or Atlas.

A restore with matching counts and a wrong value

Observation	Source	Changed restore copy
Number of tickets	4	4
Ticket 1 status	open	resolved
Ticket 3 assignment	NULL	NULL

The notebook first restores correctly, then changes only the copy. Comparing ordered IDs and values detects the wrong status.

The SQL dump lives in memory for this demonstration. It is not a durable off-machine backup.

Operational checks beyond this notebook

Concern	A relevant test in your completed course work
Access	Read as the intended user, then attempt a forbidden action
Performance	Compare the same query and returned records before and after an index
Recovery	Check the isolated restore's data and required rules
Multiple stores	Compare authoritative IDs and values with the derived copy

SQLite here does not reproduce Supabase RLS, PostgreSQL lock diagnosis, or Atlas replication. Use the appropriate earlier lab for those claims.

A readable concept guide

```
# Why an active ticket is absent from a staff report

This example uses a left join and one unassigned ticket.

## Example and result
Include the small setup, query, and result here.

## Explanation
Explain the preserved population and your changed example.
```

README structure, not a completed submission.
A reader also needs the tool, run order, cleanup, limit, and source credit.

Editing a README in GitHub

Create a repository with a README, or open one you own.
Open README.md and select the pencil icon.
Write code inside fenced blocks labeled sql or python.
Use Preview, then Commit changes with a descriptive message.

````
```sql
SELECT ticket_id FROM tickets ORDER BY ticket_id;
```
````

The individual GitHub concept lab

Choose one concept and a small example.
Change one input or query, then explain its result.
Put the example and explanation in one README.
Review the files before sharing.

Submit one public or instructor-accessible URL in Brightspace
with one sentence naming the concept and result.
A separate notebook, presentation, or full application is not required.

Day 2: Final Demonstrations

A user problem.
A model and query you can explain.
An operational decision you tested.

Use the canonical final-project assignment for requirements.
Present from your notebook, code, or prepared results. A new deck is optional.

A worked query explanation

The support manager needs to see staff workload, including zero.
I kept staff on the left and counted matching ticket IDs.
The result was Priya 1, Noah 1, Elena 0.
Ticket 3 was still active but unassigned, so I identified it
as a separate part of the backlog.

This describes the four-row review example.
Your final demonstration should describe your own project and observed output.

A resume bullet from the review example

Used SQL in a database class.

Diagnosed a zero-inclusive staff report with SQL joins and NULL-aware counts, then tested an assignment change to distinguish the assigned workload from the full backlog.

Use this wording only after doing and understanding that work.
Add the actual environment, rather than borrowing a cloud-platform claim.

An interview answer with a technical result

STAR-R part	Worked review example
Situation	The backlog count was 3, but staff counts summed to 2
Task	Explain the discrepancy without hiding a ticket
Action	Traced matched rows and tested an assignment inside a transaction
Result	The unassigned ticket explained the gap. Rollback restored the starting state
Reflection	The report needs a named unassigned category for the whole backlog

Practice individually in your notes. This is not another submission.

Answering a technical follow-up

What changes if ticket 3 is assigned to Elena?

Elena's count changes from 0 to 1.
The assigned total changes from 2 to 3.
The active total stays 3 because no ticket changed status.
Rolling back restores the earlier assignment and report.

Explain the mechanism, then identify the query or experiment that checks it.

A database choice with an explicit workload

Workload assumption	Reasonable starting design
Appointments share clients and time slots	Relational tables, keys, constraints, and transactions
Variable form snapshots are read as complete records	A document model with clear identity and validation
Both responsibilities exist	A second store only if its benefit justifies synchronization and recovery work

PostgreSQL can store JSON. MongoDB can reference documents.
A format label alone does not decide which system fits the workload.

Course work and workplace responsibilities

Work you can explain	Related workplace responsibility
Correct a report and verify its population	Application support and data analysis
Test a permission boundary	Database administration and security support
Compare plans and index costs	Backend development and database performance
Restore and check a separate target	Database operations and recovery
Load data repeatedly without duplicate identities	Data integration and engineering

These responsibilities overlap across roles. One course does not establish all requirements of a job. Compare a posting with work you actually completed.

Your next database learning project

Choose one gap in a role that interests you.
Build one small experiment that addresses it.
Describe the result and the next question it raises.

Example: automate a logical restore into a disposable target,
check known records and constraints, and report a failure clearly.
Keep expanding a workflow you can explain rather than adding tool names.